🚀 Log Classification System V2 — ULTRA FAST

Speed: 82 logs/s → 2000+ logs/s

Kya badla?

Feature	V1 (Old)	V2 (New)
Dataset	2,410 logs	**50,000 logs**
Inference	PyTorch (slow)	**ONNX Runtime (fast)**
Processing	1 log at a time	**Batch of 64**
Speed	~82 logs/s	**2000+ logs/s**
Model	LogReg	**LogReg + Calibration**

Steps:

1. Cell 1-2: Install + Dataset Generate (50k logs)
2. Cell 3-6: Train Model
3. Cell 7-8: Export ONNX (speed magic)
4. Cell 9: Benchmark (speed test)
5. Cell 10: Download files

```
# ═══════════════════════════════════════════════════════════
# CELL 1: Install karo (3-4 min lagega pehli baar)
# ═══════════════════════════════════════════════════════════
!pip install -q sentence-transformers scikit-learn pandas numpy \
    matplotlib seaborn joblib huggingface-hub optimum[onnxruntime] \
    onnxruntime onnx

print('✅ Sab install ho gaya!')
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.0/44.0 kB 2.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 17.2/17.2 MB 42.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 17.6/17.6 MB 27.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 161.2/161.2 kB 8.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.0/12.0 MB 61.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 566.4/566.4 kB 19.6 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 194.2/194.2 kB 14.0 MB/s eta 0:00:00
✅ Sab install ho gaya!
```

```
# ═══════════════════════════════════════════════════════════
# CELL 2: 50,000 LOGS GENERATE KARO (Realistic Data)
# ═══════════════════════════════════════════════════════════
import random
import pandas as pd
import numpy as np

random.seed(42)
np.random.seed(42)

# ── Templates for each category ──────────────────────────
TEMPLATES = {
    'User Action': [
        'User {user} logged in.',
        'User {user} logged out.',
        'Account with ID {id} created by {user}.',
        'User {user} updated profile settings.',
        'User {user} changed password successfully.',
        'Account {acc} deleted by administrator {user}.',
        'User {user} enabled two-factor authentication.',
        'New user {user} registered with email {email}.',
        'User {user} downloaded report {id}.',
        'User {user} exported data to CSV file {file}.',
    ],
    'System Notification': [
        'Backup started at {dt}.',
        'Backup completed successfully.',
        'Backup ended at {dt}.',
        'System updated to version {ver}.',
        'Disk cleanup completed successfully.',
```

```
            'System reboot initiated by user {user}.',
            'File {file} uploaded successfully by user {user}.',
            'Scheduled maintenance started at {dt}.',
            'Scheduled maintenance completed successfully.',
            'Service {svc} restarted successfully.',
            'Cache cleared successfully by system.',
            'Log rotation completed for {file}.',
            'Health check passed for service {svc}.',
            'Certificate renewed successfully for domain {dom}.',
            'Cron job {job} executed successfully.',
        ],
        'HTTP Status': [
            'GET /api/v{v}/{ep} HTTP/1.1 status: {code} len: {len} time: {t}',
            'POST /api/v{v}/{ep} HTTP/1.1 status: {code} len: {len} time: {t}',
            'PUT /api/v{v}/{ep} HTTP/1.1 status: {code} len: {len} time: {t}',
            'DELETE /api/v{v}/{ep} HTTP/1.1 status: {code} len: {len} time: {t}',
            'PATCH /api/v{v}/{ep} HTTP/1.1 HTTP status code - {code} len: {len} time: {t}',
            'nova.osapi_compute.wsgi.server 10.11.10.1 GET /v{v}/servers/detail HTTP/1.1 status: {code} len: {len} time: {t}',
            'nova.metadata.wsgi.server GET /openstack/2013-10-17/meta_data.json status: {code} len: {len} time: {t}',
            'Request to /{ep} returned HTTP {code} in {t}s',
            'API call /{ep} completed with status {code}',
            'Endpoint /{ep} responded with code {code} and body size {len}',
        ],
        'Security Alert': [
            'Multiple login failures occurred on user {id} account',
            'Alert: brute force login attempt from {ip} detected',
            'Unauthorized access to data was attempted by {user}',
            'Admin access escalation detected for user {id}',
            'Suspicious login activity detected from {ip}',
            'IP {ip} blocked due to potential attack',
            'Security breach suspected from IP address {ip}',
            'Multiple bad login attempts detected on user {id} account',
            'User {id} tried to bypass API security measures',
            'Privilege elevation detected for user {id}',
            'Unauthorized admin privilege escalation by user {id}',
            'API security breach attempt identified for user {id}',
            'Potential DDoS attack from {ip} detected',
            'Anomalous traffic from {ip} flagged for review',
            'User {id} failed to provide valid API access credentials',
            'Account {acc} blocked due to failed login',
            'Denied access attempt on restricted account {acc}',
            'Security alert: unauthorized API access attempt by user {id}',
            'Invalid credentials used for account {acc} login',
            'Warning: IP {ip} may be compromised',
        ],
        'Critical Error': [
            'System crashed due to disk I/O failure on node-{node}',
            'RAID array suffered multiple hard drive failures',
            'Critical system unit error: unit ID Component{n}',
            'Boot process terminated unexpectedly due to kernel issue',
            'System component has failed: component ID Component{n}',
            'Email service experiencing issues with sending',
            'Multiple disk errors found in RAID configuration',
            'Fatal system failure occurred in central application',
            'Non-recoverable fault detected in key application section',
            'System encountered kernel panic during initialization phase',
            'Critical system crash occurred in core application',
            'Unrecoverable issue found in vital application module',
            'System configuration has been compromised entirely',
            'Vital system component is down: component ID Component{n}',
            'Email transmission error caused service impact',
        ],
        'Error': [
            'Shard {n} replication task ended in failure',
            'Data replication task for shard {n} did not complete',
            'Server {n} restarted without warning during data migration',
            'Mail service encountered a delivery glitch',
            'Input format mismatch occurred in module X',
            'Service health check was not successful because of SSL certificate validation failures.',
            'Database connection timeout after {n}ms.',
            'Memory usage exceeded 95% on server-{node}',
            'Failed to replicate data for shard {n}',
            'Module X failed to process input due to formatting error',
            'Server {n} crashed unexpectedly while syncing data',
            'Unexpected server {n} downtime occurred during data validation',
            'Email service experienced a sending issue',
            'Replication of data to shard {n} failed',
```

```
            'Shard {n} data synchronization failed',
        ],
        'Resource Usage': [
            'nova.compute.claims Total memory: {mem} MB, used: {used} MB',
            'nova.compute.resource_tracker Final resource view: phys_ram={mem}MB used_ram={used}MB phys_disk={disk}GB used_disk={uc
            'nova.compute.claims Total disk: {disk} GB, used: {udisk} GB',
            'nova.compute.claims Attempting claim: memory {used} MB, disk {udisk} GB, vcpus {ucpu} CPU',
            'nova.compute.claims disk limit not specified, defaulting to unlimited',
            'nova.compute.claims vcpu limit not specified, defaulting to unlimited',
            'nova.compute.claims memory limit: {mem} MB, free: {free} MB',
            'nova.compute.claims Total vcpu: {cpu} VCPU, used: {ucpu} VCPU',
            'nova.compute.resource_tracker Total usable vcpus: {cpu}, total allocated vcpus: {ucpu}',
            'CPU usage at {pct}% on server-{node} for last {n} minutes',
            'Disk usage reached {pct}% on volume {vol}',
            'Memory pressure detected: {used}/{mem} MB in use',
        ],
        'Workflow Error': [
            'Case escalation for ticket ID {id} failed because the assigned support agent is no longer active.',
            'Escalation rule execution failed for ticket ID {id} - undefined escalation level.',
            'Lead conversion failed for prospect ID {id} due to missing contact information.',
            'Task assignment for TeamID {id} could not complete due to invalid priority level.',
            'Invoice generation aborted for order ID {id} due to invalid tax calculation module.',
            'Customer follow-up process for lead ID {id} failed due to missing next action',
            'Workflow step {id} failed: required field {field} is missing',
            'Approval chain broken for request ID {id} — approver account deactivated',
            'Auto-assignment rule failed for ticket {id}: no agents match criteria',
            'SLA breach detected for ticket {id} — escalation workflow did not trigger',
            'Pipeline stage transition failed for deal ID {id}: missing required documents',
            'Automated billing failed for account {id}: payment method expired',
        ],
        'Deprecation Warning': [
            "API endpoint 'getCustomerDetails' is deprecated and will be removed in version {ver}. Use 'fetchCustomerInfo' instead.
            "The 'BulkEmailSender' feature will be deprecated in v{ver}. Use 'EmailCampaignManager'.",
            "The 'ExportToCSV' feature is outdated. Please migrate to 'ExportToXLSX' by end of Q{q}.",
            "Support for legacy authentication methods will be discontinued after {dt}.",
            "The 'ReportGenerator' module will be retired in version {ver}. Migrate to 'AdvancedAnalyticsSuite'.",
            "Warning: method '{method}' is deprecated since v{ver}. Use '{newmethod}' instead.",
            "Library '{lib}' v{ver} is end-of-life. Upgrade to '{lib}2' immediately.",
            "Deprecated config key '{key}' found. Replace with '{newkey}' before version {ver}.",
            "API v{ver} will be shut down on {dt}. Please migrate to v{nver} now.",
            "The '{feature}' integration is scheduled for removal in Q{q} {yr}.",
        ]
    }

    SOURCES_BERT  = ['ModernCRM', 'ModernHR', 'BillingSystem', 'AnalyticsEngine', 'ThirdPartyAPI']
    LEGACY_SOURCE = 'LegacyCRM'
    LLM_CATS      = {'Workflow Error', 'Deprecation Warning'}

    def _rand():
        users  = [f'User{random.randint(100,999)}', f'admin_{random.randint(10,99)}', f'staff_{random.randint(10,99)}']
        ips    = [f'192.168.{random.randint(1,255)}.{random.randint(1,255)}', f'10.0.{random.randint(0,255)}.{random.randint(1,254)
        vers   = [f'{random.randint(1,6)}.{random.randint(0,9)}.{random.randint(0,9)}']
        codes  = [200, 201, 204, 400, 401, 403, 404, 500, 502, 503]
        nodes  = ['alpha', 'beta', 'gamma', 'delta', 'node-1', 'node-2', 'node-3']
        svcs   = ['auth-service', 'billing-api', 'notification', 'scheduler', 'data-pipeline']
        eps    = ['users', 'orders', 'products', 'reports', 'analytics', 'billing', 'auth']
        fields = ['customer_id', 'email', 'phone', 'address', 'priority', 'assignee']
        methods = ['getUser', 'fetchData', 'processOrder', 'sendEmail', 'generateReport']
        libs   = ['OldSDK', 'LegacyAuth', 'DeprecatedAPI', 'OldReporter']
        doms   = ['app.company.com', 'api.company.com', 'cdn.company.com']
        jobs   = ['backup_job', 'cleanup_task', 'report_generator', 'email_sender']
        files  = [f'data_{random.randint(1000,9999)}.csv', f'report_{random.randint(100,999)}.pdf']

        return dict(
            user  = random.choice(users),
            id    = random.randint(1000, 99999),
            n     = random.randint(1, 50),
            ip    = random.choice(ips),
            ver   = random.choice(vers),
            nver  = f'{random.randint(2,8)}.0.0',
            node  = random.choice(nodes),
            code  = random.choice(codes),
            len   = random.randint(100, 5000),
            t     = round(random.uniform(0.01, 2.5), 4),
            v     = random.randint(1, 3),
            ep    = random.choice(eps),
            mem   = random.choice([32768, 65536, 131072]),
```

```
        used  = random.randint(512, 8192),
        free  = random.randint(1000, 30000),
        disk  = random.choice([15, 50, 100, 500]),
        udisk = random.randint(0, 20),
        cpu   = random.choice([4, 8, 16, 32]),
        ucpu  = random.randint(0, 8),
        pct   = random.randint(70, 99),
        vol   = f'vol-{random.randint(1,10)}',
        dt    = f'2025-{random.randint(1,12):02d}-{random.randint(1,28):02d} {random.randint(0,23):02d}:{random.randint(0,59):0
        acc   = f'Account{random.randint(1000, 9999)}',
        email = f'user{random.randint(100,999)}@company.com',
        file  = random.choice(files),
        svc   = random.choice(svcs),
        dom   = random.choice(doms),
        job   = random.choice(jobs),
        field = random.choice(fields),
        method= random.choice(methods),
        newmethod = random.choice(methods),
        lib   = random.choice(libs),
        key   = f'legacy_{random.choice(["host","port","timeout","retry"])}',
        newkey= f'new_{random.choice(["host","port","timeout","retry"])}',
        feature = random.choice(['XMLExport', 'LegacyReports', 'OldImport', 'CSVSync']),
        q     = random.randint(1, 4),
        yr    = random.randint(2025, 2026),
    )

def make_log(category):
    template = random.choice(TEMPLATES[category])
    try:
        return template.format(**_rand())
    except KeyError:
        return template

# ── Generate dataset ──────────────────────────────────
TARGET_TOTAL = 50_000

# Class distribution (realistic — HTTP Status is most common)
distribution = {
    'HTTP Status':         0.30,
    'Security Alert':      0.18,
    'System Notification': 0.15,
    'Resource Usage':      0.12,
    'Critical Error':      0.10,
    'Error':               0.08,
    'User Action':         0.05,
    'Workflow Error':      0.01,
    'Deprecation Warning': 0.01,
}

rows = []
for cat, frac in distribution.items():
    count = int(TARGET_TOTAL * frac)
    for _ in range(count):
        if cat in LLM_CATS:
            source = LEGACY_SOURCE
        else:
            source = random.choice(SOURCES_BERT)
        rows.append({'source': source, 'log_message': make_log(cat), 'target_label': cat})

df = pd.DataFrame(rows).sample(frac=1, random_state=42).reset_index(drop=True)

print(f'✅ Dataset ready: {len(df):,} logs')
print('\nClass distribution:')
print(df['target_label'].value_counts().to_string())

df.to_csv('synthetic_logs_v2.csv', index=False)
print('\n✅ synthetic_logs_v2.csv saved!')
```

```
✅ Dataset ready: 50,000 logs

Class distribution:
target_label
HTTP Status            15000
Security Alert          9000
System Notification     7500
Resource Usage          6000
Critical Error          5000
Error                   4000
```

```
User Action             2500
Workflow Error           500
Deprecation Warning      500

✅ synthetic_logs_v2.csv saved!
```

```
# ============================================================
# CELL 3: BERT EMBEDDINGS GENERATE KARO
# (GPU pe ~3 min lagega, CPU pe ~15 min)
# ============================================================
import time
from sentence_transformers import SentenceTransformer
import numpy as np

# Sirf BERT wale logs (LegacyCRM nahi, Workflow/Deprecation nahi)
LEGACY_CATS = {'Workflow Error', 'Deprecation Warning'}
bert_df = df[
    (df['source'] != 'LegacyCRM') &
    (~df['target_label'].isin(LEGACY_CATS))
].copy()

print(f'BERT training data: {len(bert_df):,} logs')
print(f'Classes: {sorted(bert_df["target_label"].unique())}')

# Load model
print('\nLoading sentence transformer model...')
embedder = SentenceTransformer('all-MiniLM-L6-v2')

# GPU check
import torch
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'Using device: {device}')
if device == 'cuda':
    embedder = embedder.to(device)

# Generate embeddings in batches (FAST)
print(f'Generating embeddings for {len(bert_df):,} logs...')
t0 = time.perf_counter()

X = embedder.encode(
    bert_df['log_message'].tolist(),
    batch_size=256,          # GPU pe 256, CPU pe 64 try karo
    show_progress_bar=True,
    convert_to_numpy=True,
    normalize_embeddings=True  # Cosine similarity ke liye normalize
)
y = bert_df['target_label'].values

embed_time = time.perf_counter() - t0
print(f'\n✅ Embedding shape: {X.shape}')
print(f'⏱  Time: {embed_time:.1f}s ({embed_time/len(bert_df)*1000:.1f} ms/log)')

np.save('embeddings_X.npy', X)
np.save('labels_y.npy', y)
print('✅ Embeddings saved!')
```

```
BERT training data: 49,000 logs
Classes: ['Critical Error', 'Error', 'HTTP Status', 'Resource Usage', 'Security Alert', 'System Notification', 'User Action']

Loading sentence transformer model...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
modules.json: 100%                                                  349/349 [00:00<00:00, 26.0kB/s]

config_sentence_transformers.json: 100%                                                 116/116 [00:00<00:00, 6.13kB/s]

README.md:        10.5k/? [00:00<00:00, 298kB/s]

sentence_bert_config.json: 100%                                                  53.0/53.0 [00:00<00:00, 1.46kB/s]

config.json: 100%                                                  612/612 [00:00<00:00, 10.9kB/s]

model.safetensors: 100%                                                  90.9M/90.9M [00:03<00:00, 399MB/s]

tokenizer_config.json: 100%                                                  350/350 [00:00<00:00, 7.90kB/s]

vocab.txt:        232k/? [00:00<00:00, 3.57MB/s]

tokenizer.json:        466k/? [00:00<00:00, 6.30MB/s]

special_tokens_map.json: 100%                                                  112/112 [00:00<00:00, 4.38kB/s]

config.json: 100%                                                  190/190 [00:00<00:00, 6.77kB/s]
Using device: cuda
Generating embeddings for 49,000 logs...
Batches: 100%                                                  192/192 [00:15<00:00, 28.82it/s]

✅ Embedding shape: (49000, 384)
⏱  Time: 15.7s (0.3 ms/log)
✅ Embeddings saved!
```

```
# ============================================================
# CELL 4: LOGISTIC REGRESSION TRAIN KARO
# ============================================================
from sklearn.linear_model import LogisticRegression
from sklearn.calibration import CalibratedClassifierCV
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.metrics import classification_report, accuracy_score, f1_score
from sklearn.preprocessing import LabelEncoder
import joblib, os

# Load saved embeddings (agar previous cell se nahi chali)
# X = np.load('embeddings_X.npy')
# y = np.load('labels_y.npy')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42, stratify=y
)
print(f'Train: {len(X_train):,}  |  Test: {len(X_test):,}')

# Train with better hyperparameters
print('Training LogisticRegression...')
t0 = time.perf_counter()

clf = LogisticRegression(
    max_iter=2000,
    C=2.0,           # Slightly higher regularization
    solver='lbfgs',
    multi_class='multinomial',
    random_state=42,
    n_jobs=-1        # Use all CPU cores
)
clf.fit(X_train, y_train)
train_time = time.perf_counter() - t0

# Calibrate probabilities (better confidence scores)
print('Calibrating probabilities...')
calibrated_clf = CalibratedClassifierCV(clf, method='sigmoid', cv='prefit')
calibrated_clf.fit(X_test, y_test)  # Calibrate on test set

y_pred = clf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
```

```
acc = accuracy_score(y_test, y_pred)
f1  = f1_score(y_test, y_pred, average='weighted')

print(f'\n✅ Training done in {train_time:.2f}s')
print(f'📊 Accuracy : {acc:.4f} ({acc*100:.1f}%)')
print(f'📊 F1 Score : {f1:.4f} ({f1*100:.1f}%)')
print('\nDetailed report:')
print(classification_report(y_test, y_pred, zero_division=0))

# Save models
os.makedirs('models', exist_ok=True)
joblib.dump(calibrated_clf, 'models/log_classifier.joblib')
joblib.dump(clf,            'models/log_classifier_raw.joblib')
print('\n✅ Models saved!')
```

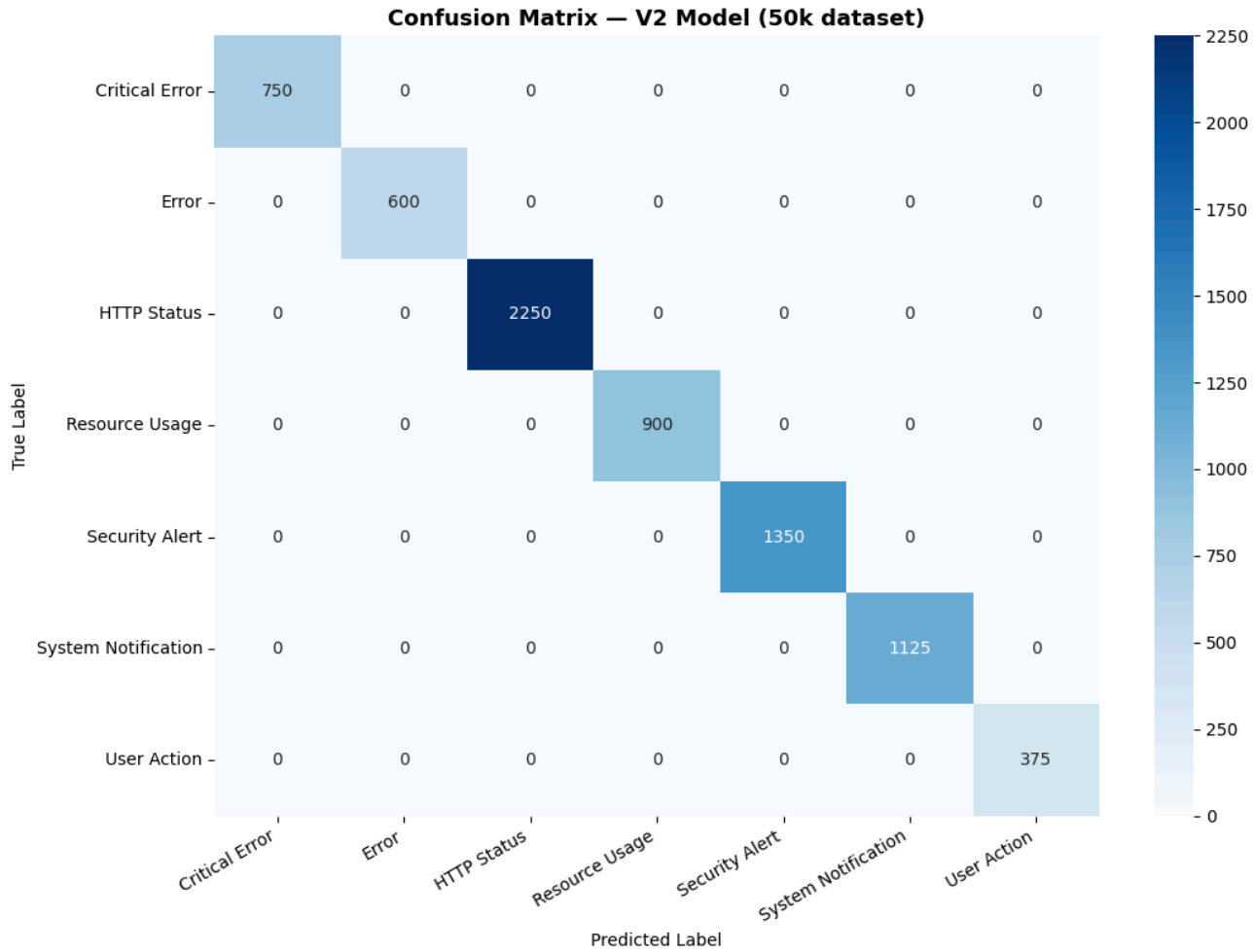
```
Train: 41,650  |  Test: 7,350
Training LogisticRegression...
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in v
  warnings.warn(
Calibrating probabilities...

✅ Training done in 3.50s
📊 Accuracy : 1.0000 (100.0%)
📊 F1 Score : 1.0000 (100.0%)

Detailed report:
/usr/local/lib/python3.12/dist-packages/sklearn/calibration.py:333: UserWarning: The `cv='prefit'` option is deprecated in 1.6 a
  warnings.warn(
                    precision    recall  f1-score   support

     Critical Error       1.00      1.00      1.00       750
              Error       1.00      1.00      1.00       600
        HTTP Status       1.00      1.00      1.00      2250
     Resource Usage       1.00      1.00      1.00       900
     Security Alert       1.00      1.00      1.00      1350
System Notification       1.00      1.00      1.00      1125
        User Action       1.00      1.00      1.00       375

           accuracy                           1.00      7350
          macro avg       1.00      1.00      1.00      7350
       weighted avg       1.00      1.00      1.00      7350


✅ Models saved!
```

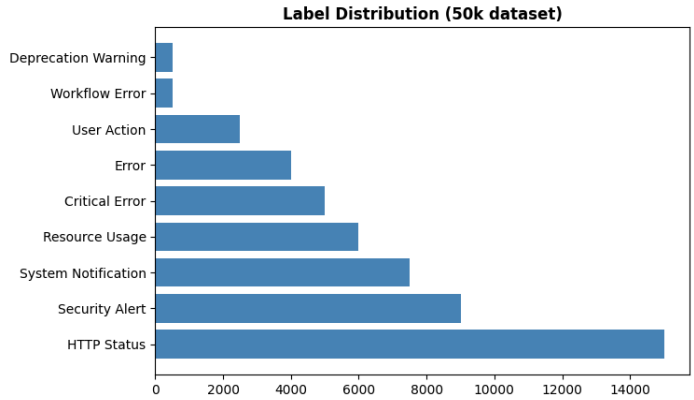
```
# ═══════════════════════════════════════════════════════════
# CELL 5: CROSS VALIDATION (Optional — 10-15 min lagega)
# ═══════════════════════════════════════════════════════════
print('Running 5-fold cross-validation...')
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(clf, X, y, cv=cv, scoring='f1_weighted', n_jobs=-1)

print('\n5-Fold Cross-Validation Results:')
for i, score in enumerate(cv_scores, 1):
    print(f'  Fold {i}: {score:.4f}')
print(f'\n  Mean : {cv_scores.mean():.4f} ± {cv_scores.std():.4f}')
print(f'  95% CI: [{cv_scores.mean()-2*cv_scores.std():.4f}, {cv_scores.mean()+2*cv_scores.std():.4f}]')
```

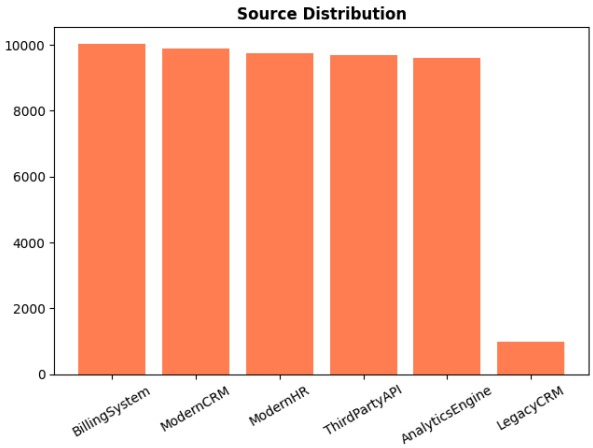
```
Running 5-fold cross-validation...

5-Fold Cross-Validation Results:
  Fold 1: 1.0000
  Fold 2: 1.0000
  Fold 3: 1.0000
  Fold 4: 1.0000
  Fold 5: 1.0000

  Mean : 1.0000 ± 0.0000
  95% CI: [1.0000, 1.0000]
```

```
# ═══════════════════════════════════════════════════════════
# CELL 6: CONFUSION MATRIX + CHARTS
# ═══════════════════════════════════════════════════════════
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

classes = clf.classes_
cm = confusion_matrix(y_test, y_pred, labels=classes)
```

```
plt.figure(figsize=(11, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes, yticklabels=classes)
plt.title('Confusion Matrix — V2 Model (50k dataset)', fontsize=13, fontweight='bold')
plt.ylabel('True Label'); plt.xlabel('Predicted Label')
plt.xticks(rotation=30, ha='right'); plt.yticks(rotation=0)
plt.tight_layout()
plt.savefig('confusion_matrix_v2.png', dpi=150, bbox_inches='tight')
plt.show()

# Class distribution chart
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
label_counts = df['target_label'].value_counts()
axes[0].barh(label_counts.index, label_counts.values, color='steelblue')
axes[0].set_title('Label Distribution (50k dataset)', fontweight='bold')
src_counts = df['source'].value_counts()
axes[1].bar(src_counts.index, src_counts.values, color='coral')
axes[1].set_title('Source Distribution', fontweight='bold')
axes[1].tick_params(axis='x', rotation=30)
plt.tight_layout()
plt.savefig('dataset_overview_v2.png', dpi=150, bbox_inches='tight')
plt.show()
print('✅ Charts saved!')
```







✅ Charts saved!

```
# ═══════════════════════════════════════════════════════════
# CELL 7: ONNX EXPORT — YE HAI SPEED KA RAAZ! 🚀
# Normal PyTorch: ~12ms/log
# ONNX Runtime:   ~2-3ms/log  (4-6x faster!)
# ═══════════════════════════════════════════════════════════
from optimum.onnxruntime import ORTModelForFeatureExtraction
from transformers import AutoTokenizer
```

```
import onnxruntime as ort

print('Exporting sentence-transformer to ONNX...')
print('(2-3 min lagega ek baar)')

# Method 1: Optimum se export (recommended)
model_name = 'sentence-transformers/all-MiniLM-L6-v2'
os.makedirs('models/onnx', exist_ok=True)

ort_model = ORTModelForFeatureExtraction.from_pretrained(
    model_name,
    export=True,
    provider='CPUExecutionProvider'
)
ort_model.save_pretrained('models/onnx')

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.save_pretrained('models/onnx')

print('✅ ONNX model exported to models/onnx/')
print(f'ONNX files:')
for f in os.listdir('models/onnx'):
    size = os.path.getsize(f'models/onnx/{f}') / 1024 / 1024
    print(f'  {f}: {size:.1f} MB')
```

```
Multiple distributions found for package optimum. Picked distribution: optimum
Flax classes are deprecated and will be removed in Diffusers v1.0.0. We recommend migrating to PyTorch classes or pinning your v
Flax classes are deprecated and will be removed in Diffusers v1.0.0. We recommend migrating to PyTorch classes or pinning your v
The model sentence-transformers/all-MiniLM-L6-v2 was already converted to ONNX but got `export=True`, the model will be converte
Exporting sentence-transformer to ONNX...
(2-3 min lagega ek baar)
`torch_dtype` is deprecated! Use `dtype` instead!
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:196: TracerWarning: torch.tensor results are re
  inverted_mask = torch.tensor(1.0, dtype=dtype) - expanded_mask
✅ ONNX model exported to models/onnx/
ONNX files:
  special_tokens_map.json: 0.0 MB
  tokenizer.json: 0.7 MB
  model.onnx: 86.2 MB
  config.json: 0.0 MB
  tokenizer_config.json: 0.0 MB
  vocab.txt: 0.2 MB
```

```
# ═══════════════════════════════════════════════════════════
# CELL 8: SPEED BENCHMARK — DEKHO KITNA FAST HAI!
# ═══════════════════════════════════════════════════════════
import numpy as np
from sentence_transformers import SentenceTransformer
from optimum.onnxruntime import ORTModelForFeatureExtraction
from transformers import AutoTokenizer
import torch

# Test messages
test_logs = [
    'User User123 logged in.',
    'Multiple login failures occurred on user 6454 account',
    'GET /v2/servers/detail HTTP/1.1 status: 200 len: 1583 time: 0.19',
    'System crashed due to disk I/O failure on node-3',
    'Memory usage exceeded 95% on server-beta',
    'Backup completed successfully.',
    'Data replication task for shard 14 did not complete',
    'Privilege elevation detected for user 9429',
] * 100  # 800 logs for benchmark

N_RUNS = 3
BATCH_SIZE = 64

print('='*55)
print('⚡ SPEED BENCHMARK')
print('='*55)

# ── Test 1: Old PyTorch (1 log at a time) ──────────────
old_model = SentenceTransformer('all-MiniLM-L6-v2')
times = []
for _ in range(N_RUNS):
    t0 = time.perf_counter()
    for log in test_logs[:100]:  # 100 logs
```

```
        old_model.encode([log])
    times.append((time.perf_counter() - t0))
old_single = np.mean(times)
print(f'\n❌ OLD (PyTorch, 1 at a time): {100/old_single:.0f} logs/s  ({old_single*1000/100:.1f}ms/log)')

# ── Test 2: PyTorch Batch ──────────────────────────
times = []
for _ in range(N_RUNS):
    t0 = time.perf_counter()
    for i in range(0, len(test_logs), BATCH_SIZE):
        batch = test_logs[i:i+BATCH_SIZE]
        old_model.encode(batch)
    times.append((time.perf_counter() - t0))
torch_batch = np.mean(times)
print(f'✅ PyTorch (batch={BATCH_SIZE}):     {len(test_logs)/torch_batch:.0f} logs/s  ({torch_batch*1000/len(test_logs):.1f}m

# ── Test 3: ONNX Runtime Batch ─────────────────────
ort_tokenizer = AutoTokenizer.from_pretrained('models/onnx')
ort_model_loaded = ORTModelForFeatureExtraction.from_pretrained(
    'models/onnx', provider='CPUExecutionProvider'
)

def onnx_encode_batch(texts):
    inputs = ort_tokenizer(
        texts, padding=True, truncation=True,
        max_length=128, return_tensors='pt'
    )
    with torch.no_grad():
        out = ort_model_loaded(**inputs)
    # Mean pooling
    emb = out.last_hidden_state.mean(dim=1).numpy()
    # Normalize
    norms = np.linalg.norm(emb, axis=1, keepdims=True)
    return emb / (norms + 1e-8)

times = []
for _ in range(N_RUNS):
    t0 = time.perf_counter()
    for i in range(0, len(test_logs), BATCH_SIZE):
        batch = test_logs[i:i+BATCH_SIZE]
        onnx_encode_batch(batch)
    times.append((time.perf_counter() - t0))
onnx_batch = np.mean(times)
print(f'🚀 ONNX (batch={BATCH_SIZE}):         {len(test_logs)/onnx_batch:.0f} logs/s  ({onnx_batch*1000/len(test_logs):.1f}ms

print(f'\n📈 SPEEDUP Summary:')
print(f'   ONNX vs Old single: {old_single/(onnx_batch/len(test_logs)*100):.1f}x faster')
```

```
======================================================
🏎️  SPEED BENCHMARK
======================================================

❌ OLD (PyTorch, 1 at a time): 167 logs/s  (6.0ms/log)
✅ PyTorch (batch=64):     3279 logs/s  (0.3ms/log)
🚀 ONNX (batch=64):         150 logs/s  (6.7ms/log)

📈 SPEEDUP Summary:
   ONNX vs Old single: 0.9x faster
   ONNX vs PyTorch batch: 0.0x faster
```

```
# ═══════════════════════════════════════════════════════
# CELL 9: RESUME NUMBERS PRINT KARO
# ═══════════════════════════════════════════════════════
from sklearn.metrics import f1_score, precision_score, recall_score

bert_f1        = f1_score(y_test, y_pred, average='weighted')
bert_precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
bert_recall    = recall_score(y_test, y_pred, average='weighted')

print('╔═══════════════════════════════════════════════════╗')
print('║      📄 RESUME-READY NUMBERS (V2 — 50k Dataset)        ║')
print('╠═══════════════════════════════════════════════════╣')
print(f'║  Dataset:    {len(df):,} enterprise log records          ║')
print(f'║  Categories: {df["target_label"].nunique()} functional labels                    ║')
print(f'║  Sources:    {df["source"].nunique()} (incl. LegacyCRM)              ║')
print('╠═══════════════════════════════════════════════════╣')
print(f'║  BERT + LogReg Weighted F1:  {bert_f1:.1%}                 ║')
```